

E-Way Intelligence - Authentication System

Complete Technical Documentation & 30-Minute Presentation Guide

Project: E-Way Intelligence System

Repository: https://github.com/Ani032006/eway_project

Last updated: June 2026

Commits covered: 8d3e4de, ddd8f16, a1247b2

Table of Contents

1. [Executive Summary](#1-executive-summary)
 2. [What We Built - Feature List](#2-what-we-built--feature-list)
 3. [System Architecture](#3-system-architecture)
 4. [Technology Stack](#4-technology-stack)
 5. [Repository Structure](#5-repository-structure)
 6. [File-by-File Explanation](#6-file-by-file-explanation)
 7. [User Flows (Step by Step)](#7-user-flows-step-by-step)
 8. [API Reference](#8-api-reference)
 9. [Security Design](#9-security-design)
 10. [Environment Configuration](#10-environment-configuration)
 11. [How to Run & Demo](#11-how-to-run--demo)
 12. [30-Minute Presentation Script](#12-30-minute-presentation-script)
 13. [Common Questions (Q&A Prep)](#13-common-questions-qa-prep)
-

1. Executive Summary

The E-Way Intelligence System is a government-style tax intelligence dashboard for monitoring E-Way bill movement. Before officers can access sensitive bill data, they must prove their identity through a secure authentication layer.

We implemented a full-stack authentication system with:

- **Officer registration** with mandatory **email OTP verification**
- **JWT (JSON Web Token)** based login sessions
- **Forgot password** and **reset via email link**
- **Gmail SMTP** email delivery using **Nodemailer** and **App Passwords**
- **Role-based access control** (Admin vs Officer)
- **Startup diagnostics** so SMTP and environment issues are visible immediately

The backend is a Node.js + Express REST API backed by MongoDB. The frontend is React with client-side route

guards. Authentication is stateless - the server does not store sessions; instead it issues signed JWT tokens that the browser stores in localStorage.

2. What We Built - Feature List

Feature	Description	Status
Officer Registration	Name, email, password, state, department	Done
Email OTP Verification	6-digit code, 5-minute expiry	Done
JWT Login	Bearer token, 7-day default expiry	Done
Forgot Password	Sends reset link without revealing if email exists	Done
Reset Password	Token + email + new password	Done
Change Password	Protected endpoint for logged-in users	Done
Welcome Email	Sent after successful OTP verification	Done
Admin Seeding	Auto-creates admin account on startup	Done
Admin Panel APIs	Pending approvals, officer list	Done
SMTP Diagnostics	Startup check + `test-email` endpoint + CLI script	Done
Dev Mode Fallback	OTP/reset URL logged to console when SMTP fails	Done
Frontend Route Guards	Protected routes, public-only auth pages	Done

3. System Architecture

```
BROWSER (React Frontend)
Login Register Forgot Password Reset Password Dashboard
localStorage: ewb_token, ewb_user

HTTP (fetch)
Authorization: Bearer <JWT>

EXPRESS API SERVER (port 5000)
/api/auth/* authRoutes authController
/api/ewb/* ewbRoutes (protected bill data)

Middleware chain: authenticate requireAdmin (where needed)

MongoDB (Users) Gmail SMTP (smtp.gmail.com)
password (bcrypt hash) Nodemailer OTP / Reset / Welcome
otp, reset tokens Port 587 (STARTTLS) or 465 (SSL)
```

Request flow for a protected API call

1. User logs in server returns `{ token, user }`
 2. Frontend saves token to `localStorage` as `ewb\_token`
 3. Every API call in `api.js` reads the token and sends `Authorization: Bearer <token>`
 4. `authenticate` middleware verifies JWT signature and expiry
 5. Middleware loads user from MongoDB and attaches to `req.user`
 6. Controller runs business logic
-

4. Technology Stack

Layer	Technology	Purpose
Runtime	Node.js	Backend JavaScript runtime
Framework	Express.js	HTTP routing and middleware
Database	MongoDB + Mongoose	User persistence, schema validation
Auth	jsonwebtoken (JWT)	Stateless session tokens
Password hashing	bcryptjs	One-way password encryption
Email	Nodemailer	SMTP email sending
Email provider	Gmail + App Password	Reliable free SMTP for development
Crypto	Node.js `crypto` module	Reset token generation and SHA-256 hashing
Frontend	React + React Router	SPA with protected routes
Config	dotenv	Environment variables from `.env`

5. Repository Structure

The Git repository lives in the frontend/ folder (monorepo layout):

```
frontend/  
  backend/ # Node.js API  
  server.js # Entry point, startup checks  
  config/db.js # MongoDB connection  
  models/User.js # User schema  
  controllers/  
  authController.js # All auth business logic  
  routes/  
  authRoutes.js # URL controller mapping  
  middleware/  
  auth.js # JWT verification  
  roles.js # Admin/officer guards  
  utils/  
  emailService.js # Nodemailer + HTML templates  
  otp.js # OTP and reset token helpers  
  validators.js # Input validation  
  seedAdmin.js # Admin account bootstrap  
  frontend/ # React app  
  src/  
  App.jsx # Route guards  
  services/api.js # HTTP client + auth functions
```

```
pages/  
Login/  
Register/  
ForgotPassword/  
ResetPassword/  
test-email.js # Standalone SMTP diagnostic CLI  
docs/  
AUTHENTICATION_SYSTEM_GUIDE.md # This document
```

6. File-by-File Explanation

6.1 `backend/server.js` - Application Entry Point

Purpose: Boots the Express server, wires routes, and runs startup health checks.

Key code blocks:

```
require("dotenv").config({ path: path.join(__dirname, ".env") });
```

Loads environment variables from backend/.env before anything else runs.

```
app.use("/api/auth", authRoutes);  
app.use("/api/ewb", ewbRoutes);
```

Mounts authentication at /api/auth and E-Way bill APIs at /api/ewb.

```
function checkEnvConfig() { ... }
```

Prints a formatted banner checking JWT_SECRET, MONGO_URI, and SMTP credentials. Warns if placeholders are still in use.

```
const start = async () => {  
  checkEnvConfig();  
  await connectDB();  
  await seedAdmin();  
  loadPincodes();  
  await verifySmtpConnection();  
  app.listen(PORT, ...);  
};
```

Startup sequence:

1. Validate environment
2. Connect MongoDB (exit if fail)
3. Ensure admin account exists
4. Load pincode reference data
5. Test SMTP connection before accepting traffic
6. Start listening on port 5000

Why this matters: Problems with email or database are surfaced at boot time, not when a user tries to register.

6.2 `backend/config/db.js` - Database Connection

Purpose: Single function to connect Mongoose to MongoDB.

```
const conn = await mongoose.connect(process.env.MONGO_URI);
```

Uses the connection string from .env. On failure, logs a clear message and calls process.exit(1) so the server does not run in a broken state.

6.3 `backend/models/User.js` - User Data Model

Purpose: Defines what a user looks like in MongoDB and how passwords are handled.

Schema fields:

Field	Type	Meaning
`name`	String	Officer full name
`email`	String	Unique, lowercased login identifier
`password`	String	Bcrypt-hashed (never stored plain text)
`role`	enum	"officer" or "admin"
`state`	String	Jurisdiction state
`department`	String	Government department
`isVerified`	Boolean	`true` after OTP verification
`approved`	Boolean	Admin approval flag (auto-true on OTP verify for officers)
`otp`	String	Current 6-digit verification code
`otpExpires`	Date	When OTP becomes invalid
`resetPasswordToken`	String	SHA-256 hash of reset token (not plain token)
`resetPasswordExpires`	Date	Reset link expiry (15 minutes)

Password hashing (pre-save hook):

```
userSchema.pre("save", async function (next) {
  if (!this.isModified("password")) return next();
  this.password = await bcrypt.hash(this.password, 10);
  next();
});
```

Every time password changes, bcrypt hashes it with cost factor 10 before MongoDB stores it. The original password is never persisted.

Password comparison:

```
userSchema.methods.comparePassword = async function (candidatePassword) {
  return bcrypt.compare(candidatePassword, this.password);
};
```

Login uses bcrypt.compare - a timing-safe comparison against the stored hash.

6.4 `backend/utils/validators.js` - Input Validation

Purpose: Centralised validation so controllers stay clean.

- `validateEmail(email)` - checks presence and regex format
- `validatePassword(password)` - minimum 6 characters
- `validateRegisterBody(body)` - validates all registration fields including state and department

Returns null if valid, or an error string if invalid. Controllers return 400 Bad Request with that message.

6.5 `backend/utils/otp.js` - OTP & Reset Token Generation

Purpose: Cryptographic helpers for verification codes.

```
function generateOtp() {  
  return String(Math.floor(100000 + Math.random() * 900000));  
}
```

Generates a random 6-digit number (100000-999999).

```
function generateResetToken() {  
  const token = crypto.randomBytes(32).toString("hex");  
  return { token, hashed: hashToken(token) };  
}
```

For password reset:

- `token` - 64-character hex string sent in the email URL
- `hashed` - SHA-256 hash stored in the database

Security note: Only the hash is stored. If the database is compromised, attackers cannot use the hash to reset passwords - they need the original token from the email.

6.6 `backend/utils/emailService.js` - Email Delivery Engine

Purpose: Sends all transactional emails via Gmail SMTP using Nodemailer.

Strategy

1. **SMTP only** (Gmail App Password)
2. Try configured port first (default **587**), fallback to **465**
3. If SMTP fails or is unconfigured **dev mode** (log email content to console)

getSmtptCredentials()

Reads SMTP_USER and SMTP_PASS from .env. Returns null if missing triggers dev mode.

buildTransporter(port)

Creates a Nodemailer transport with:

- `secure: port === 465` - SSL on 465, STARTTLS on 587
- `family: 4` - **force IPv4** (fixes IPv6 routing issues on some networks)
- Generous timeouts (15s) for slow connections
- `rejectUnauthorized: false` - allows self-signed certs on restricted networks

diagnoseSMTPError(err, port)

Translates raw SMTP errors into human-readable fixes:

- **535** wrong App Password or 2FA not enabled
- **ECONNREFUSED / ETIMEDOUT** ISP blocking SMTP, try hotspot or port 465

sendMail({ to, subject, html, text })

Core send function used by all email types:

1. Check credentials
2. Try primary port, then fallback port
3. On permanent 5xx errors, stop retrying
4. Return `{ devMode: false, provider: "smtp", port }` on success
5. Return `{ devMode: true, smtpError }` on failure

Email templates

Function	When sent	Content
----- ----- -----		
`sendOTPEmail`	Registration / resend OTP	Branded HTML with large 6-digit OTP, 5-min warning
`sendPasswordResetEmail`	Forgot password	Button + plain-text link, 15-min warning
`sendWelcomeEmail`	After OTP verified	Welcome message + dashboard link
`sendTestEmail`	Diagnostic only	Confirms SMTP is working

All templates use inline CSS (email-client compatible) with the E-Way brand colours (#004848, #FFFFCC).

verifySmtpConnection()

Called at server startup. Uses transporter.verify() to test credentials without sending an email. Logs success or detailed failure diagnosis.

6.7 `backend/utils/seedAdmin.js` - Admin Bootstrap

Purpose: Ensures an admin account always exists when the server starts.

```
const email = (process.env.ADMIN_EMAIL || "admin@gmail.com").toLowerCase();
const password = process.env.ADMIN_PASSWORD || "admin123";
```

- If admin does not exist creates one with `role: "admin"`, `isVerified: true`, `approved: true`
- If admin exists and `SYNC\_ADMIN\_PASSWORD=true` updates password from `.env`

This lets admins log in immediately without going through OTP registration.

6.8 `backend/middleware/auth.js` - JWT Authentication Middleware

Purpose: Protects routes that require a logged-in user.

```
const header = req.headers.authorization;
if (!header || !header.startsWith("Bearer ")) {
  return res.status(401).json({ error: "Authentication required" });
}
const token = header.split(" ")[1];
```

Extracts the JWT from the Authorization: Bearer <token> header.

```
const decoded = jwt.verify(token, JWT_SECRET);
const user = await User.findById(decoded.id).select("-password -otp -resetPasswordToken");
```

Verifies signature and expiry, then loads the user without sensitive fields.

```
if (!user.isVerified) {
  return res.status(403).json({ error: "Email not verified" });
}
req.user = user;
next();
```

Rejects unverified accounts even if they have a token. Attaches req.user for downstream handlers.

JWT payload structure:

```
{ "id": "<mongodb_id>", "email": "officer@example.com", "role": "officer" }
```

Expiry: JWT_EXPIRES_IN env var, default "7d" (7 days).

6.9 `backend/middleware/roles.js` - Role-Based Guards

Purpose: Restricts endpoints to specific roles.

- `requireAdmin` - only `role === "admin"` can proceed
- `requireOfficer` - only `role === "officer"` can proceed

Used after authenticate in the middleware chain:

```
router.get("/pending", authenticate, requireAdmin, ctrl.getPendingRegistrations);
```

6.10 `backend/controllers/authController.js` - Business Logic

Purpose: Contains every authentication operation. This is the largest and most important file.

Helper functions

signToken(user) - Creates JWT with id, email, role.

publicUser(user) - Returns safe user object (no password, no OTP).

issueOtp(user) - Generates OTP, saves to DB with 5-minute expiry, sends email.

register - POST /api/auth/register

Flow:

1. Validate body (name, email, password, state, department)
 2. Block registration of `admin@gmail.com` (reserved for seeded admin)
 3. If email exists and already verified `409 Conflict`
 4. If email exists but NOT verified update details and resend OTP (allows retry)
 5. If new email create user with `isVerified: false`, `approved: false`
 6. Issue OTP via email
 7. In dev/non-production, include `devOtp` in response for testing
-

sendOtp - POST /api/auth/send-otp

Flow:

1. Validate email
2. Find user - `404` if not found
3. Reject if already verified
4. Generate and send new OTP

Used by the Register page "Resend OTP" button.

verifyOtp - POST /api/auth/verify-otp

Flow:

1. Validate email and 6-digit OTP
2. Check OTP exists and has not expired
3. Compare OTP (plain text comparison - acceptable because OTP is short-lived and single-use)
4. Set `isVerified: true`, `approved: true`
5. Clear OTP fields
6. Send welcome email (non-blocking - failure does not block response)
7. Return JWT token + user object

Important: User is logged in immediately after OTP verification - no separate login step required.

login - POST /api/auth/login

Flow:

1. Validate email and password
 2. Find user - generic `401 Invalid credentials` (does not reveal if email exists)
 3. `comparePassword` with bcrypt
 4. If not verified `403` with `requiresVerification: true` and email (frontend redirects to OTP step)
 5. If officer and not approved `403 Account pending approval`
 6. Return token + user
-

forgotPassword - POST /api/auth/forgot-password

Flow:

1. Validate email
2. If user not found OR not verified return **same success message** (prevents email enumeration)
3. Generate reset token, store **hash** in DB with 15-minute expiry
4. Build URL: `{CLIENT\_URL}/reset-password?token=...&email=...`
5. Send reset email
6. In dev mode, include `devResetUrl` in response

Security: Attackers cannot discover which emails are registered.

resetPassword - POST /api/auth/reset-password

Flow:

1. Validate token, email, new password
 2. Find user where `resetPasswordToken === hashToken(token)` AND expiry > now
 3. If not found `400 Invalid or expired reset token`
 4. Set new password (bcrypt hook hashes it), clear reset fields
 5. Return success message
-

changePassword - POST /api/auth/change-password (protected)

Flow:

1. Requires `authenticate` middleware
 2. Verify current password
 3. Ensure new password is different
 4. Save new password
-

getProfile - GET /api/auth/profile (protected)

Returns req.user (already loaded and sanitised by middleware).

Admin endpoints

Endpoint	Purpose
GET /pending	List verified but unapproved officers
PATCH /approve/:id	Approve an officer
GET /officers	List all officers

testEmail - POST /api/auth/test-email

Diagnostic endpoint. Sends a test email and returns detailed success/failure with SMTP troubleshooting hints.

6.11 `backend/routes/authRoutes.js` - Route Map

Purpose: Maps HTTP methods and paths to controller functions and middleware.

PUBLIC (no token needed):

- POST /register
- POST /send-otp
- POST /verify-otp
- POST /login
- POST /forgot-password
- POST /reset-password
- POST /test-email

PROTECTED (Bearer token):

- POST /change-password
- GET /profile

ADMIN ONLY (Bearer token + admin role):

- GET /pending
- PATCH /approve/:id
- GET /officers

All paths are prefixed with /api/auth in server.js.

6.12 `frontend/src/services/api.js` - HTTP Client

Purpose: Single place for all backend communication.

request(endpoint, options) - Core fetch wrapper:

1. Reads `ewb\_token` from `localStorage`
2. Adds `Authorization: Bearer <token>` if token exists
3. Sets `Content-Type: application/json`
4. Parses JSON response
5. Throws structured errors with `err.status` and `err.data`

Exports named functions: login, register, sendOtp, verifyOtp, forgotPassword, resetPassword, changePassword, getProfile, logout, etc.

logout() - Removes ewb_token and ewb_user from localStorage.

6.13 `frontend/src/App.jsx` - Route Guards

Purpose: Controls which pages users can access based on login state.

isAuthenticated() - Checks localStorage for a valid token (rejects placeholder immediate_access_token).

ProtectedRoute - Redirects to /login if not authenticated. Optional adminOnly prop redirects non-admins to dashboard.

PublicOnlyRoute - Redirects logged-in users away from login/register pages to dashboard.

Route map:

Path	Guard	Page
-----	-----	-----
/login	PublicOnly	Login
/register	PublicOnly	Register
/forgot-password	PublicOnly	ForgotPassword
/reset-password	PublicOnly	ResetPassword
/dashboard	Protected	Dashboard
/bill/:id	Protected	BillDetail

6.14 `frontend/src/pages/Login/Login.jsx`

Purpose: Officer login form.

Key behaviour:

- Calls `login(email, password)`
- On success: saves `token` and `user` to `localStorage`, navigates to dashboard
- On `requiresVerification` error: shows "Go to OTP Verification" button that navigates to Register with email pre-filled
- Links to forgot password and registration

6.15 ``frontend/src/pages/Register/Register.jsx``

Purpose: Two-step registration wizard.

Step 1 - Form: Collects name, email, password, state, department. Calls `register()`.

Step 2 - OTP: Shows 6-digit input, countdown timer (5 minutes), verify and resend buttons.

Special features:

- Can start directly on OTP step if redirected from Login with unverified email
- Shows ``devOtp`` in success message during development
- On successful verification: saves token, auto-redirects to dashboard

6.16 ``frontend/src/pages/ForgotPassword/ForgotPassword.jsx``

Purpose: Request password reset link.

- Calls ``forgotPassword(email)``
- Shows generic success message (same whether email exists or not)
- In dev mode, displays ``devResetUrl`` inline for testing

6.17 ``frontend/src/pages/ResetPassword/ResetPassword.jsx``

Purpose: Set new password using token from email.

- Reads ``token`` and ``email`` from URL query params (``?token=...&email=...``)
- If from email link: hides token/email fields (pre-filled)
- Password strength indicator and confirm-password matching
- Calls ``resetPassword({ token, email, password })``
- Redirects to login on success

6.18 ``test-email.js`` - CLI Diagnostic Script

Purpose: Test SMTP without starting the full server.

```
node test-email.js your.email@gmail.com
```

Steps:

1. Load ``backend/.env``
2. Run ``verifySmtpConnection()``
3. Send ``sendTestEmail(recipient)``

4. Exit with code 0 (success) or 1 (failure)

Use this when debugging Gmail App Password or network blocking issues.

7. User Flows (Step by Step)

Flow A: New Officer Registration

User fills registration form

POST /api/auth/register

- Validate fields
- Create user (isVerified=false)
- Generate 6-digit OTP (5 min TTL)
- Send OTP email via SMTP

User enters OTP on Register page

POST /api/auth/verify-otp

- Validate OTP + expiry
- Set isVerified=true, approved=true
- Send welcome email (async)
- Return JWT token

Frontend saves token Dashboard

Flow B: Login (Verified User)

User enters email + password

POST /api/auth/login

- bcrypt.compare(password)
- Check isVerified
- Sign JWT

localStorage token + user Dashboard

Flow C: Login (Unverified User)

POST /api/auth/login 403 requiresVerification

Frontend shows "Go to OTP Verification"

Navigate to /register with email pre-filled

User completes OTP step

Flow D: Forgot / Reset Password

User enters email on Forgot Password page

POST /api/auth/forgot-password

- Generate 32-byte random token
- Store SHA-256 hash in DB (15 min TTL)
- Email link: /reset-password?token=...&email=...

User clicks link in email

Reset Password page (token + email from URL)

POST /api/auth/reset-password

- Find user by email + hashed token + expiry
- Set new password (bcrypt hashed)
- Clear reset fields

Redirect to Login

Flow E: Protected API Access

Frontend: Authorization: Bearer eyJhbG...

authenticate middleware

- jwt.verify(token, JWT_SECRET)
- Load user from MongoDB
- Attach req.user

Controller executes (e.g. getProfile, upload bills)

8. API Reference

Base URL: <http://localhost:5000/api/auth>

Public Endpoints

POST /register

```
// Request
{
  "name": "Officer Name",
  "email": "officer@example.com",
  "password": "secure123",
  "state": "Maharashtra",
  "department": "GST Intelligence"
}

// Response 201
{
  "message": "Registration started. Enter the OTP sent to your email.",
  "email": "officer@example.com",
  "requiresVerification": true,
  "devOtp": "482910" // only in dev/non-production
}
```

POST /send-otp

```
{ "email": "officer@example.com" }
```

POST /verify-otp

```
// Request
{ "email": "officer@example.com", "otp": "482910" }

// Response 200
{
  "message": "Email verified. Your account is now active.",
  "token": "eyJhbGciOiJIUzI1NiIs... ",
  "user": { "id": "...", "name": "...", "role": "officer", ... }
}
```

POST /login

```
// Request
{ "email": "officer@example.com", "password": "secure123" }

// Response 200
{ "token": "...", "user": { ... } }
```

POST /forgot-password

```
{ "email": "officer@example.com" }
// Always returns same message regardless of whether email exists
```

POST /reset-password

```
{
  "token": "abc123...",
  "email": "officer@example.com",
  "password": "newpassword123"
}
```



```
}
```

```
##### POST /test-email
```

```
{ "email": "test@example.com" }
```

Protected Endpoints (Header: `Authorization: Bearer <token>`)

```
##### POST /change-password
```

```
{
  "currentPassword": "old123",
  "newPassword": "new456"
}
```

```
##### GET /profile
```

Returns current user object.

Admin Endpoints

```
##### GET /pending
```

```
##### PATCH /approve/:id
```

```
##### GET /officers
```

9. Security Design

| Measure | Implementation |

|-----|-----|

| Password storage | bcrypt with cost factor 10 |

| Session management | Stateless JWT (no server-side sessions) |

| Token in transit | HTTPS recommended in production; Bearer header |

| OTP expiry | 5 minutes, cleared after use |

| Reset token storage | SHA-256 hash only; 15-minute expiry |

| Email enumeration | Forgot password returns same message always |

| Login errors | Generic "Invalid credentials" |

| Sensitive fields | Excluded from API responses (`-password -otp`) |

| Role separation | Middleware enforces admin vs officer |

| Admin email reserved | Cannot register as admin@gmail.com |

| Dev fallbacks | OTP/reset URL in API only in non-production |

Production recommendations:

- Set a strong random `JWT\_SECRET`
- Use HTTPS
- Set `NODE\_ENV=production` to hide `devOtp` and `devResetUrl`
- Rotate Gmail App Password periodically

10. Environment Configuration

Create backend/.env:

```
# Server
PORT=5000
NODE_ENV=development

# Database
MONGO_URI=mongodb://127.0.0.1:27017/eway_intelligence

# JWT
JWT_SECRET=your_long_random_secret_here
JWT_EXPIRES_IN=7d

# Admin bootstrap
ADMIN_EMAIL=admin@gmail.com
ADMIN_PASSWORD=admin123

# Frontend URL (for reset links and welcome emails)
CLIENT_URL=http://localhost:5173

# Gmail SMTP
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your.email@gmail.com
SMTP_PASS=xxxx xxxx xxxx xxxx
EMAIL_FROM=E-Way Intelligence <your.email@gmail.com>
```

Gmail App Password setup:

1. Enable 2-Step Verification on Google Account
2. Go to Security App Passwords
3. Generate password for "Mail"
4. Paste 16-character password into `SMTP\_PASS` (spaces optional)

11. How to Run & Demo

Start everything

```
cd frontend
npm run install:all # first time only
npm run dev # starts API (5000) + React (5173)
```

Demo script for presentation

1. **Show startup logs** - environment check + SMTP verification
2. **Register** a new officer at `http://localhost:5173/register`
3. **Show OTP email** in inbox (or console dev mode)

4. ****Verify OTP**** auto-login to dashboard
 5. ****Logout**** try login with wrong password (generic error)
 6. ****Forgot password**** show email with reset link
 7. ****Reset password**** login with new password
 8. ****Admin login**** `admin@gmail.com` / `admin123`
 9. ****Test SMTP**** - `node test-email.js your@email.com`
-

12. 30-Minute Presentation Script

Use this as a spoken guide. Approximate timing per section.

[0:00 - 2:00] Introduction

"Good morning/afternoon. Today I will present the authentication system we built for the E-Way Intelligence platform - a government tax intelligence dashboard for monitoring E-Way bill movement.

>

Before officers can access sensitive transportation and billing data, they must prove their identity. We implemented a complete authentication layer: registration with email verification, secure login, password recovery, and role-based access for admins and officers.

>

The system uses industry-standard technologies: Node.js and Express on the backend, React on the frontend, MongoDB for storage, JWT for sessions, and Gmail SMTP for transactional emails."

[2:00 - 5:00] Problem Statement & Requirements

"Our requirements were:

>

1. *Only verified government officers should access the dashboard.*
2. *Registration must confirm the officer owns the email address - we chose OTP verification.*
3. *Passwords must never be stored in plain text.*
4. *Officers who forget passwords should reset via email, not admin intervention.*
5. *Admins need a separate elevated role for user management.*
6. *The system must work in development even when SMTP is blocked - hence our dev mode fallback.*

>

We structured the repo as a monorepo inside the `frontend/` folder: `backend/` for the API, `frontend/` for React, sharing one Git repository."

[5:00 - 9:00] Architecture Overview

(Show architecture diagram from Section 3)

>

"The architecture follows a classic three-tier pattern.

>

The React frontend stores a JWT in browser localStorage. Every API call sends it as a Bearer token.

>

The Express server has two route groups: `/api/auth` for authentication and `/api/ewb` for bill data. Auth routes are mostly public; bill routes require authentication.

>

MongoDB stores user records with bcrypt-hashed passwords. Gmail SMTP delivers OTP codes, reset links, and welcome emails via Nodemailer.

>

*The key design choice is **stateless authentication**: the server does not maintain session tables. The JWT is self-contained - it carries the user ID, email, and role, signed with a secret key."*

[9:00 - 14:00] Backend Deep Dive

"Let me walk through the backend files.

>

***server.js** is the entry point. On startup it runs four checks: environment variables, MongoDB connection, admin seeding, and SMTP verification. This means we discover configuration problems before any user hits the system.*

>

***User.js** defines our schema. Important fields: `isVerified` for OTP completion, `approved` for admin approval, `otp` and `otpExpires` for verification codes, and `resetPasswordToken` which stores only a SHA-256 hash - never the plain reset token.*

>

The password pre-save hook automatically bcrypt-hashes any password change. Controllers never touch hashing directly.

>

***authController.js** is the brain. It has nine public operations and three admin operations.*

>

Registration is smart: if someone registered before but never verified, they can re-register with the same email and get a fresh OTP - we update their record instead of rejecting them.

>

OTP verification immediately logs the user in - we return a JWT right away so they do not need a separate login

step.

>

Forgot password uses a security pattern called **email enumeration prevention**: we always return the same message whether the email exists or not.

>

emailService.js handles all emails. We use Gmail App Passwords - not the regular Gmail password - because Google requires App Passwords for third-party SMTP access when 2FA is enabled.

>

We try port 587 first, then fall back to 465, and force IPv4 to avoid network routing issues. If everything fails, dev mode logs the OTP to the console so development can continue."

[14:00 - 17:00] Security Walkthrough

"Security was not an afterthought.

>

Passwords: bcrypt, cost factor 10, automatic via Mongoose hook.

>

Reset tokens: 32 random bytes, SHA-256 hashed in database, 15-minute expiry. Even a database leak does not expose usable reset links.

>

JWT: signed with `JWT_SECRET`, default 7-day expiry, verified on every protected request.

>

Login errors are intentionally vague - 'Invalid credentials' - so attackers cannot probe which emails are registered.

>

The `authenticate` middleware strips password, OTP, and reset token from the user object before attaching it to the request.

>

Admin accounts cannot be created through registration - `admin@gmail.com` is blocked, and the real admin is seeded from environment variables on startup."

[17:00 - 21:00] Frontend Integration

"On the frontend, **api.js** is our single HTTP client. It automatically attaches the JWT from localStorage to every request.

>

App.jsx implements route guards. `ProtectedRoute` wraps the dashboard - unauthenticated users redirect to

login. `PublicOnlyRoute` wraps auth pages - logged-in users redirect to the dashboard.

>

The **Register page** is a two-step wizard: form, then OTP. It includes a live countdown timer matching the 5-minute server expiry. If a user tries to login before verifying, the login page detects `requiresVerification` and offers a one-click jump back to OTP entry.

>

Forgot Password shows a success state regardless of outcome. **Reset Password** reads the token from the URL query string when the user clicks the email link - they only need to enter a new password.

>

After OTP verification or login, we store two items in `localStorage`: `ewb_token` for API auth and `ewb_user` for displaying name and role in the UI."

[21:00 - 25:00] Live Demo Narration

(Perform live demo following Section 11)

>

"Watch the server console on startup - you will see the environment check banner and SMTP verification.

>

I will register as a new officer... the API creates the user and sends a 6-digit OTP.

>

Here is the email in my inbox - branded HTML template with the OTP prominently displayed.

>

I enter the OTP... verification succeeds, welcome email fires in the background, and I receive a JWT. The app redirects to the dashboard.

>

Now I log out and demonstrate forgot password... the reset email arrives with a button linking to our React reset page.

>

I set a new password and log in successfully.

>

Finally, admin login uses the seeded account - no OTP required because admin is pre-verified."

[25:00 - 28:00] Challenges & Solutions

"We faced several real-world challenges:

>

****1. SMTP blocked by ISP/college network**** - Solved with port 465 fallback, IPv4 forcing, and dev mode console logging.

>

****2. Gmail authentication failures (535 error)**** - Documented in ``diagnoseSMTPError``: must use App Password, not login password; 2FA must be enabled.

>

****3. Unverified users trying to login**** - Backend returns structured error; frontend guides them to OTP step.

>

****4. Email delivery uncertainty**** - Added ``verifySmtplibConnection`` at startup, ``POST /test-email`` endpoint, and ``test-email.js`` CLI script.

>

****5. Monorepo structure**** - Backend lives inside ``frontend/backend/`` with shared npm scripts (``npm run dev`` starts both)."

[28:00 - 30:00] Conclusion & Future Work

"To summarise: we built a production-ready authentication system with email OTP verification, JWT sessions, secure password reset, Gmail SMTP integration, role-based access, and comprehensive diagnostics.

>

All code is committed and pushed to GitHub on the ``main`` branch.

>

Possible future enhancements:

- Refresh tokens for longer sessions without re-login
- Rate limiting on OTP and login endpoints
- Account lockout after failed attempts
- Email verification link as alternative to OTP
- HTTPS deployment with secure cookie-based tokens

>

Thank you. I am happy to take questions."

13. Common Questions (Q&A Prep)

Q: Why JWT instead of server-side sessions?

A: JWT is stateless - the server does not need a session store. It scales easily and works well with SPAs. Trade-off: tokens cannot be revoked until expiry without a blacklist.

Q: Why is OTP stored in plain text but reset token is hashed?

A: OTP is 6 digits, expires in 5 minutes, and is single-use. Reset tokens are 64 hex characters with a 15-minute window - higher value target, so we hash them.

Q: What happens if SMTP is down?

A: Dev mode logs OTP/reset URL to the server console. API responses include devOtp or devResetUrl in non-production. User flows still work for testing.

Q: Can anyone become admin?

A: No. Registration blocks admin@gmail.com. Admin is created only via seedAdmin.js from ADMIN_EMAIL in .env.

Q: How long is the user logged in?

A: JWT default expiry is 7 days (JWT_EXPIRES_IN). After that, jwt.verify fails and user must log in again.

Q: Is the frontend security enough?

A: Frontend route guards are UX only. Real security is enforced by backend authenticate middleware on every protected API call.

Appendix: Git Commit History (Auth-related)

```
| Commit | Message |
|-----|-----|
| `8d3e4de` | Implement authentication, email verification OTP, and role-based route guards |
| `ddd8f16` | Force IPv4 connection in nodemailer SMTP configuration |
| `a1247b2` | Add SMTP diagnostics, welcome emails, and startup env verification |
```

End of document. To export as PDF: open this file in VS Code Markdown PDF extension, or paste into Google Docs / Word Export as PDF.